```
      (compared using ==)
    # ./spec/password_strength_validator_spec.rb:11:in `block (2 levels)
    in <top (required)>'

Finished in 0.01002 seconds (files took 0.08962 seconds to load)
2 examples, 1 failure

Failed examples:

rspec ./spec/password_strength_validator_spec.rb:9 #
PasswordStrengthValidator accepts passwords 8 characters or longer
```

The test is broken, even though the code works correctly. What should have been an easy configuration change is now requiring us to spend time fighting a broken test.

Decouple Your Tests from Incidental, Changeable Details



In *Test Doubles: Mocks, Stubs, and Others*, on page 67, you constructed your test environment using a fake Ledger that returned a canned "valid" or "invalid" status. That way, you could check how your API reported results, without getting caught up in validation rules that are likely to change.

To avoid brittle specs, use test doubles to decouple them from validation rules, configuration, and other ever-changing specifics of your application.

Instead, we can choose to configure the password requirements explicitly as part of the test environment:

15-using-test-doubles-effectively/02/password_s … lidator/spec/password_strength_validator_spec.rb

```
RSpec.describe PasswordStrengthValidator do
  before do
    allow(Acme::Config).to receive(:min_password_length).and_return(6)
  end

  it 'rejects passwords shorter than the configured length' do
    validator = PasswordStrengthValidator.new('a8E^r')
    expect(validator.strong_enough?).to eq false
  end

  it 'accepts passwords that satisfy the configured length' do
    validator = PasswordStrengthValidator.new('a8E^rd')
    expect(validator.strong_enough?).to eq true
  end
end
```

The spec will continue to pass, no matter how many times we tweak the app's configuration. We're no longer coupled to the current value of min_password_length.